PYTHON FOR BEGINNERS · SESSION 4

Conditional Statements

Make your programs decide with if, elif, else and nested conditions.

Recap

Ask the class:

- What is the difference between `=` and `==`?
- What is the output of `"Apple" == "apple"`?
- What does `age > 18 and marks > 75` give when both parts are True?

Why do we need conditions?

So far every line of our programs ran, **one after another**. Real programs need to **make decisions**: allow the login only if the password is right, give a discount only if the bill is large enough.

A **condition** is an expression that gives **True or False**, such as `age >= 18`. A conditional statement runs a block of code **only when the condition is True**.

STATEMENT	USE IT WHEN
`if`	one thing needs a check
`if` … `else`	there are two ways to go
`if` … `elif` … `else`	there are several ways to go
nested `if`	a decision depends on an earlier decision

The if statement

Syntax

```
if condition:
    statement
```

age.py

```
age = 20
if age >= 18:
    print("You can vote")
print("Done")
```

Colon and indentation

- The line ends with a **colon** (`:`).
- The code that belongs to the `if` is **indented**, normally by **4 spaces**.
- Python uses **indentation instead of brackets**. Every indented line is inside the block.
- The first line that is **not indented** (`print("Done")`) runs every time.

A block can have many lines

```
marks = 72
if marks >= 40:
    print("Pass")
    print("Well done")
print("Result checked")
```

Ask

What is printed if `marks = 30`?

Forgetting the indentation

Try it: read the error, then indent the print line

```
age = 20
if age >= 18:
print("You can vote")
```

The if ... else statement

`else` runs when the condition is **False**. Exactly **one** of the two blocks runs.

```
number = 7
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

- `else` has **no condition** of its own.

- `else` also ends with a **colon** and has an **indented block**.
- `else` lines up with the `if` it belongs to.

Try it yourself

Your turn: change the age, then press Run

```
age = 16
if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible to vote")
```

The if … elif … else ladder

Use `elif` (short for **else if**) when there are **more than two choices**. Python checks from the top and runs the **first block whose condition is True**, then skips the rest.

Grade ladder

```
marks = 82
if marks >= 90:
    print("Grade A+")
elif marks >= 75:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
elif marks >= 40:
    print("Grade C")
else:
    print("Fail")
```

- You can have **as many** `elif` blocks as you need.
- `else` is **optional** and always comes **last**.
- Only **one** block in the whole ladder runs.

Ask

What grade does `marks = 75` give? What about `marks = 74`?

Order matters

Put the **most specific** condition first. Here the first test is True for 82, so the better grade is never reached.

Run it and spot the bug

```
marks = 82
if marks >= 40:
    print("Grade C")
elif marks >= 75:
    print("Grade A")
```

Separate ifs or elif?

```
x = 15
print("Separate ifs:")
if x > 10:
    print("more than 10")
if x > 5:
    print("more than 5")
print("With elif:")
if x > 10:
    print("more than 10")
elif x > 5:
    print("more than 5")
```

Separate `if` statements are **all checked**. An `elif` is checked **only if the earlier ones were False**.

Nested conditions

An `if` **inside another `if`** is called a nested condition. The inner check happens **only if the outer one is True**.

```
age = 25
has_id = True
if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("Show your ID")
else:
    print("Too young")
```

- Each level is indented **4 more spaces**.
- An `else` belongs to the `if` at the **same indentation**.

Nested or and?

When you only need **both** checks to pass, `and` is shorter and easier to read.

```
age = 25
has_id = True
if age >= 18 and has_id:
    print("Entry allowed")
else:
    print("Not allowed")
```

Use nesting when you need a **different message for each failure**, as in the first example.

Largest of three numbers

```
a, b, c = 12, 45, 30
if a >= b:
    if a >= c:
        print(a, "is largest")
    else:
        print(c, "is largest")
else:
    if b >= c:
        print(b, "is largest")
    else:
        print(c, "is largest")
```

Ask

Can you write the same program with `and` and `elif`? Which is easier to read?

Writing good conditions

Combining with and, or, not

```
age = 30
income = 25000
if age >= 18 and income > 20000:
    print("Loan approved")
if age < 18 or age > 60:
    print("Special ticket")
else:
    print("Normal ticket")
```

Chained comparison

Python lets you write a range check the way you would in maths.

```
age = 30
print(18 <= age <= 60)
print(18 <= age and age <= 60)
```

Checking membership

```
letter = "e"
if letter in "aeiou":
    print("Vowel")
else:
    print("Consonant")
```

Truthy and falsy values

A condition does not have to be a comparison. **Empty or zero values count as False**, and everything else counts as True.

```
name = ""
if name:
    print("Hello", name)
else:
    print("Name is empty")
```

Comparing text safely

Text comparison is **case-sensitive**. Convert first with `lower()` so `YES`, `Yes` and `yes` all work.

```
answer = "YES"
if answer.lower() == "yes":
    print("Confirmed")
else:
    print("Cancelled")
```

One-line if (conditional expression)

When a decision only **picks one of two values**, you can write it on **one line**.

```
age = 20
status = "adult" if age >= 18 else "minor"
print(status)
```

Pattern

`value_if_true if condition else value_if_false`. Keep it for **short, simple** choices. Use a normal `if` block for anything longer.

Common mistakes

MISTAKE	EXAMPLE	FIX
`=` instead of `==`	`if age = 18:`	`if age == 18:`
Missing colon	`if age >= 18`	`if age >= 18:`
Missing indentation	body on the same level as `if`	indent the body by 4 spaces
Uneven indentation	3 spaces on one line, 4 on the next	use the same amount for the whole block
Text compared with a number	`"20" >= 18`	`int("20") >= 18`
`elif` or `else` with no `if`	`else:` on its own	start with `if`

Assignment or comparison?

Try it: read the error, then fix it

```
age = 18
if age = 18:
    print("Eighteen")
```

Text and numbers

`input()` always gives text, so convert before comparing.

Try it: read the error, then fix it with int()

```
age = "20"     # imagine this came from input()
if age >= 18:
    print("Adult")
```

Run this one in VS Code or a terminal

```
age = int(input("Enter your age: "))
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Practice questions

Basic questions

1. Take an age and print whether the person is eligible to vote.
2. Take a number and print whether it is positive, negative or zero.
3. Take a number and print whether it is even or odd.
4. Take marks and print the grade: A for 90 or more, B for 75 or more, C for 60 or more, D for 40 or more, otherwise Fail.
5. Take a letter and print whether it is a vowel or a consonant.

Harder questions

6. Take three numbers and print the largest.
7. Take a year and check whether it is a leap year. A year is a leap year if it is divisible by 4 and not by 100, or if it is divisible by 400.
8. Ticket price: free for age below 5, half price from 5 to 12, full price above 12. Take the age and print the price.
9. Simple calculator: take two numbers and an operator (`+`, `-`, `*`, `/`) and print the result. Print a message if the user divides by zero.
10. Login check: take a username and password and print "Login successful" only if both are correct.

Find the output

Work out the output first, then press Run to check

```
x = 7
if x > 5:
    if x > 10:
        print("A")
    else:
        print("B")
else:
    print("C")
```

Try the leap year question

Your turn: finish the program, change the year and press Run

```
year = 2024
# print "Leap year" or "Not a leap year"
```

Homework and wrap-up

- Take marks in **3 subjects**, work out the **average**, and print the **grade** and whether the student has passed (average of at least 40).
- Write an **electricity bill** program: up to 100 units cost 5 per unit, and each unit above 100 costs 8.
- Take the **three sides** of a triangle. Check that it is a valid triangle (each side is shorter than the sum of the other two), then print whether it is equilateral, isosceles or scalene.
- Revise the **indentation rules** and why the **order of `elif` blocks matters**.

Next class

Loops: `while` and `for`.